

features

LIST

mutable
Ordered
Indexing
Slicing

Initialization

- `lst = []`
- `lst = list()`
- `lst1 = list("hello")`

↳ `["h", "e", "l", "l", "o"]`

- `lst2 = [10, 20, "hello", 4.7]`

Accessing the elements (Indexing)

→ `str2 = ["h", "e", "l", "l", "o"] ["h", "e", "l", "l", "o"]`
 `print(str2[2])` ↗ `ll`

• Indexing

lst1 = [[1, 2, 3, [4, 5]], [6, [7, 8]], 9, 10, 11]

lst1

[1, 2, 3, [4, 5]]	[6, [7, 8]]	9	10	11
-5	-4	-3	-2	-1
0	1	2	3	4

print(lst1[0][1]) → 2

print(lst1[0][3]) → [4, 5]

print(lst1[0][3][1]) → 5

print(lst1[-5][3][1]) → 5

print(lst1[-5][-1][-1]) → 5

• slicing

lst1 = ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]

lst1[2:7]

• print(lst1[2:7])

↳ ["c", "d", "e", "f", "g"]

• print(lst1[2:9:2])

↳ ["c", "e", "g", "i"]

• print(lst1[-4:8])

↳ ["g", "h"]

⇒ Initialization,
Indexing, Adding,
Slicing, Removing,
Packing, Comparing

★ Adding Element in the list

• append()

l1 = [1, 2, 3, 4]

l1.append(5)

print(l1)

↳ [1, 2, 3, 4, 5]

l1.append("hello")

↳ [1, 2, 3, 4, 5, "hello"]

• insert()

l1 = [1, 2, 3, 4]

l1.insert(1, "hello")

↳ [1, "hello", 2, 3, 4]

• extend()

iterable,

l1 = [1, 2, 3, 4]

l1.extend("hello")

↳ [1, 2, 3, 4, "h",

"e", "l", "l", "o"]


```
l1 = [1, 2, 3, 4]
l1.append([7, 8, 9])
print(l1)
```

⇒ [1, 2, 3, 4, [7, 8, 9]]

```
l1 = [1, 2, 3, 4]
l1.extend([7, 8, 9])
print(l1)
```

⇒ [1, 2, 3, 4, 7, 8, 9]

• Modify

```
l2 = [7, 8, 9, 10]
l2[0] = 11
print(l2)
↳ [11, 8, 9, 10]
```

List as input from user:

```
l1 = input("Enter values").split()
```

Enter values: 1 2 3 4 5 6

```
@ l1 = []
```

```
count = (input("Enter no. of values"))
```

```
for i in range(count):
```

```
l1.append(int(input("Enter values")))
```

```
l1 = ["Rohit", "Sachin", "Dhoni", "Virat", "Jadeja"]
```

Accepts string (Value to Value Replacement)

```
l1[0] = "Raina"
print(l1)
```

```
↳ ["Raina", "Sachin", "Dhoni", "Virat", "Jadeja"]
```

Accepts list

(List to List Replacement)

```
l1[2:4] = "Raina"
print(l1)
```

```
↳ ["Rohit", "Sachin", "R", "a", "i", "n", "a", "Jadeja"]
```

Replace the list

(List to List Replacement)

```
l1[2:4] = ["Raina"]
print(l1)
```

```
↳ ["Rohit", "Sachin", "Raina", "Jadeja"]
```


Remove with help of values.

• `remove()`

`l1 = [10, 20, 30, 40]`

• `l1.remove(30)`
`print(l1)`

↳ `[10, 20, 40]`

• `l1.remove(60)`
`print(l1)`

↳ `Error`

⇒ List packing & unpacking

`lst1 = [10, 20, 30]`

• `a, b, c = lst1`

`print(a) → 10`

`print(b) → 20`

`print(c) → 30`

• `a, *b, c = [10, 20, 30, 40, 50]`

`print(a) → 10`

`print(c) → 50`

`print(b) → [20, 30, 40]`

Remove with help of Index.

• `del`

`l1 = [10, 20, 30, 40]`

• `del l1[1]`
`print(l1)`

↳ `[10, 30, 40]`

• `del l1[1:3]`
`print(l1)`

↳ `[10, 40]`

• `del l1[:]`
`print(l1)`

↳ `[]`

• `del l1`
`print(l1)`

↳ `Error`

Empty whole list

• `clear()`

`l1 = [10, 20, 30, 40]`

`l1.clear()`

`print(l1)`

↳ `[]`

if the index not declared, it simply pops the value from the end.

• `pop()`

`l1 = [10, 20, 30, 40]`

• `a = l1.pop(1)`

`print(a) → 20`

`print(l1)`

↳ `[10, 30, 40]`

• `b = l1.pop()`

`print(b) → 40`

`print(l1)`

↳ `[10, 20, 30]`

★ List comprehension

↳ concise way to create a list.

• `lst1 = []`

for `i` in `range(1, 101)`:

`lst1.append(i)`

⇒ `[1, 2, 3, ..., 99, 100]`

• `lst1 = [i for i in range(1, 101)]`

⇒ `[1, 2, 3, ..., 99, 100]`

• `lst2 = [i*2 for i in range(1, 11)]`

`print(lst2)`

↳ `[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]`

$l1 = [i+5 \text{ for } i \text{ in range } (1,6)]$

$\rightarrow [6, 7, 8, 9, 10]$

$l1 = [i \text{ for } i \text{ in range } (1,11) \text{ if } i \% 2 \neq 0]$

$\rightarrow [2, 4, 6, 8, 10]$

$l2 = [i+5 \text{ if } i \% 2 \neq 0 \text{ else } i+10 \text{ for } i \text{ in range } (1,5)]$

$\rightarrow [11, 7, 13, 9]$

$l3 = [(i,j) \text{ for } i \text{ in range } (1,3) \text{ for } j \text{ in range } (1,3)]$

$\rightarrow [(1,1), (1,2), (2,1), (2,2)]$

- ~~sort()~~
- sorted()
- reverse()
- reversed()

$\Rightarrow$ sort(), reverse() are inplace methods

$\Rightarrow$ Change applied on same list.

$\Rightarrow$ sorted(), reversed() makes a copy of actual list with changes applied.

$\Rightarrow$ No changes in actual list.

$\Rightarrow$ Changes only in copied list.

sort()

$l1 = [9, 2, 7, 3, 4, 6, 11]$

$l1.sort()$ inplace

$print(l1)$

$\rightarrow [2, 3, 4, 6, 7, 9, 11]$

O/P

$l1.sort(reverse=True)$

$\rightarrow [11, 9, 7, 6, 4, 3, 2]$

O/P

reverse()

$l1 = [11, 6, 4, 3, 7, 2, 9]$

$l1.reverse()$

$print(l1) \Rightarrow [9, 2, 7, 3, 4, 6, 11]$

sorted()
 $l1 = [9, 2, 7, 3, 4, 6, 11]$

$a = sorted(l1)$

$print(l1) \rightarrow [9, 2, 7, 3, 4, 6, 11]$

$print(a) \rightarrow [2, 3, 4, 6, 7, 9, 11]$

reversed()

$l1 = [11, 6, 4, 3, 7, 2, 9]$

$x = reversed(l1)$

$print(l1) \rightarrow [11, 6, 4, 3, 7, 2, 9]$

$print(x) \rightarrow [9, 2, 7, 3, 4, 6, 11]$